SAÉ 2.03 : Rapport Technique

Table des matières

1. Semaine 7 : Préparation de la machine virtuelle et OS de base . . . 2
   1.1. Préparation de la machine virtuelle . . . 2
      1.1.1. Création et caractéristiques de la VM . . . 2
      1.1.2. Installation de l'OS de base . . . 2
      1.1.3. Préparation du système et Suppléments invités . . . 3
   1.2. Réponses aux questions techniques (Semaine 07) . . . 4
      1.2.1. Configuration matérielle dans VirtualBox (Question 1) . . . 4
      1.2.2. Installation OS de base (Question 2) . . . 4
      1.2.3. Sudo et Suppléments invités (Questions 3 & 4) . . . 5
   1.3. À propos de la distribution Debian (Question 4.2) . . . 5
   1.4. Installation préconfigurée (Question 5) . . . 6
      1.4.1. Ajustement de la pré-configuration . . . 6
      1.4.2. Notre configuration automatisée . . . 6
2. Semaine 8 : Comparatif Markdown vs AsciiDoc . . . 8
   2.1. Étude comparative des formats à balisage léger . . . 8
3. Semaine 10 : Étude des applications clientes (Git) . . . 12
   3.1. Configuration globale de Git . . . 12
   3.2. Concepts de base (Question 1) . . . 12
   3.3. Protocoles réseau pour Git (Question 2) . . . 12
   3.4. Manipulation pratique de Git (Question 3) . . . 13
   3.5. Travail collaboratif (Question 4) . . . 16
   3.6. Les interfaces graphiques pour git (Question 5) . . . 16
   3.7. Installons autre chose et comparons (Questions 6 & 7) . . . 16
4. Semaine 11 : Installation de Forgejo . . . 19
   4.1. Question 1 : À propos de Forgejo . . . 19
   4.2. Question 2 : Architecture et protocoles réseau . . . 19
   4.3. Question 3 : Base de données . . . 20
   4.4. Question 4 : Gestion du service systemd . . . 20
   4.5. Question 5 : Sécurité . . . 21
   4.6. Question 6 : Tests d'utilisation approfondis . . . 21
   4.7. Question 7 : Intégration et livraison continues (Pour aller plus loin) . . . 22

1. Semaine 7 : Préparation de la machine virtuelle et OS de base

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

Présentation

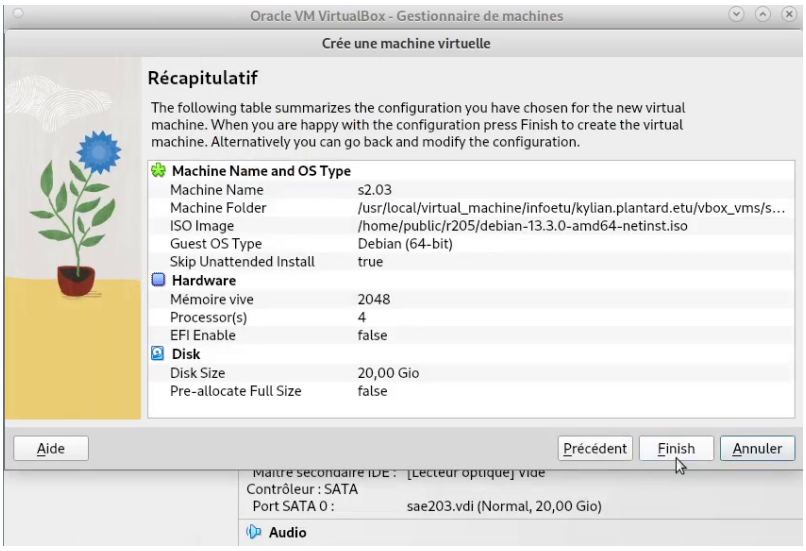
Ce rapport retrace l'installation, la configuration et l'automatisation d'une machine virtuelle sous Debian 13, ainsi que les réponses aux questions techniques associées.

1.1. Préparation de la machine virtuelle

1.1.1. Création et caractéristiques de la VM

Nous avons mis en place une machine virtuelle dans VirtualBox en respectant les spécifications suivantes :

- **Nom de la machine** : `s2.03`
- **Type de système** : Linux
- **Version** : Debian (64-bit)
- **Mémoire vive (RAM)** : 2048 Mo
- **Disque dur** : 20 Go (une seule partition de la taille totale)

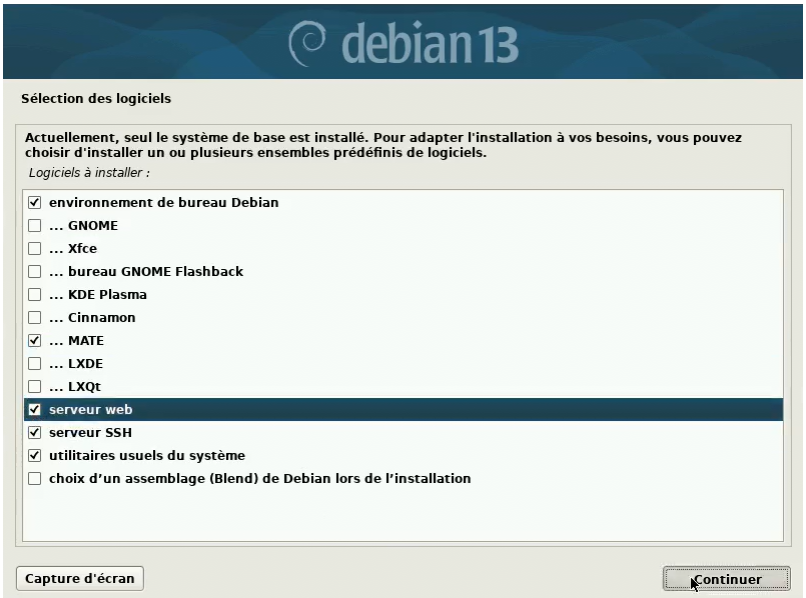


1.1.2. Installation de l'OS de base

Lors de l'installation du système d'exploitation à partir de l'image ISO, nous avons appliqué les paramètres suivants :

1. **Nom de la machine** : `serveur`

2. **Miroir réseau** : `http://debian.polytech-lille.fr`
3. **Comptes créés** :
   - Administrateur : `root`
   - Utilisateur standard : `user`
4. **Sélection des logiciels** :
   - Environnement de bureau Debian
   - MATE
   - Serveur web
   - Serveur SSH
   - Utilitaires usuels du système



1.1.3. Préparation du système et Suppléments invités

Afin de faciliter l'administration du système, nous avons accordé les droits `sudo` à l'utilisateur `user` via la console tty1 (`Ctrl` + `Alt` + `F1`) en étant connecté en tant que `root` :

```
usermod -aG sudo user
```

man 8 usermod [1]

Nous avons ensuite inséré le CD virtuel des **Additions Invité** (Guest Additions) depuis le menu de VirtualBox, puis nous l'avons monté et installé en ligne de commande :

```
sudo mount /dev/cdrom /mnt
sudo /mnt/VBoxLinuxAdditions.run
```

man 8 mount [2]

Un redémarrage a permis d'activer le redimensionnement dynamique de la fenêtre.

1.2. Réponses aux questions techniques (Semaine 07)

1.2.1. Configuration matérielle dans VirtualBox (Question 1)

Question	Réponse justifiée
Que signifie "64-bit" ?	Cela désigne l'architecture du processeur capable de traiter des données par blocs de 64 bits [3]. C'est l'architecture la plus courante pour les systèmes modernes.
Configuration réseau par défaut ?	Par défaut, VirtualBox émule un adaptateur réseau ethernet `Intel PRO/1000 MT Desktop` avec un cable virtuel connecté. `PCNet FAST III` est l'adaptateur par défaut pour les anciennes machines, mais les Debian utilisent le nouvel adaptateur [4]
Pourquoi 2048 Mo de RAM ? Et avec 512 Mo ?	2048 Mo permettent de faire tourner confortablement l'installation ainsi que l'environnement graphique MATE [5]. Avec seulement 512 Mo, le système utiliserait massivement la partition d'échange (Swap), rendant la machine extrêmement lente, et l'interface graphique pourrait même refuser de se lancer par manque de mémoire.
Mode réseau par défaut ?	Le mode NAT. Il isole la VM tout en lui donnant accès à Internet via la connexion de la machine hôte [6].
Adresse IP de la VM ?	`ip address` [7] L'adresse IP est `10.0.2.15`.
Adresse de la passerelle ?	`ip route` La passerelle par défaut est `10.0.2.2`. Elle correspond au routeur virtuel créé par VirtualBox pour le mode NAT.

1.2.2. Installation OS de base (Question 2)

Question	Réponse justifiée
Fichier iso bootable ?	C'est une image disque (une copie exacte d'un CD/DVD) contenant les fichiers nécessaires pour amorcer (booter) un ordinateur et installer un système d'exploitation [8].
MATE et GNOME ?	Ce sont des Environnements de Bureau (Desktop Environments) pour Linux. Ils fournissent l'interface graphique (fenêtres, icônes, barres des tâches). GNOME est plus lourd et moderne, MATE est un dérivé plus léger [9].
Serveur mandataire (proxy) ?	C'est un serveur qui agit comme intermédiaire entre les requêtes d'un client (notre VM) et un autre serveur (Internet). Il peut filtrer, mettre en cache ou sécuriser les connexions [10].

Question	Réponse justifiée
Serveur web ?	Un logiciel (comme Apache ou Nginx) qui répond aux requêtes HTTP/HTTPS pour distribuer des pages web à des clients (navigateurs) [11].
Serveur SSH installé ?	Le paquet installé est `openssh-server`. On vérifie son état avec la commande `systemctl status ssh`.
Test de connexion SSH ?	Commande : `ssh user@10.0.2.15`. **Problème rencontré** : La connexion échoue ("Timeout"). **Pourquoi ?** Parce que le mode réseau NAT de VirtualBox crée un réseau séparé de celui de l'hôte. Il faut configurer une redirection de ports (Port Forwarding) dans VirtualBox pour que cela fonctionne [12].

1.2.3. Sudo et Suppléments invités (Questions 3 & 4)

- **Appartenance aux groupes** : Pour savoir à quels groupes appartient l'utilisateur `user`, on utilise la commande `groups user` [13] ou `id user` [14].
- **Différence entre `su` et `sudo`** :
  - `su` [15] (substitute user) permet de changer d'utilisateur (souvent pour devenir `root`) et requiert le mot de passe de l'utilisateur cible.
  - `sudo` [16] (substitute user do) permet d'exécuter une seule commande avec les privilèges d'un autre utilisateur (généralement `root`), mais nécessite le mot de passe de l'utilisateur courant.
- **Version du noyau Linux** : `uname -r` [17] : `6.12.73+deb13-amd64`
- **Utilité des suppléments invités** : Ils optimisent la VM [18]. Deux raisons de les installer :
  - Permettre le redimensionnement dynamique de la fenêtre (ajustement automatique de la résolution).
  - Activer le presse-papiers partagé entre la machine hôte et la VM.
- **Commande `mount`** : En général, elle sert à attacher un système de fichiers (disque, clé USB) à l'arborescence du système (point de montage). Dans notre cas, elle a servi à rendre accessible le contenu du lecteur CD-ROM virtuel pour y exécuter le script d'installation.

1.3. À propos de la distribution Debian (Question 4.2)

Pour répondre à ces questions, nous avons consulté la documentation officielle de Debian. . **Le Projet Debian et son nom** : C'est une association d'individus qui ont pour cause commune de créer un système d'exploitation libre. Le nom "Debian" a été imaginé par son créateur, Ian Murdock, en combinant le prénom de sa petite amie de l'époque (Debra) et le sien (Ian).
What is the Debian Project? . **Durées de prise en charge** : * *Minimale (Standard)* : Environ 3 ans (1 an après la sortie de la version stable suivante). * *LTS (Long Term Support)* : 5 ans au total. Debian LTS * *ELTS (Extended LTS)* : Jusqu'à 10 ans (géré par une organisation tierce, Freexian). Debian ELTS . **Mises à jour de sécurité** : L'équipe de sécurité Debian fournit un support pour la version stable pendant environ 1 an après la sortie d'une nouvelle version stable. Debian Release Lifecycle . **Versions activement maintenues** : Il y en a au minimum 3. Leurs noms génériques sont : *Stable*,

Testing (en test), et *Unstable* (instable, surnommée *Sid*). On peut aussi compter *Oldstable*. Debian Release Lifecycle . **Origine des noms de code** : Tous les noms de code des versions Debian proviennent des personnages des films d'animation *Toy Story* (ex. : Bullseye, Bookworm, Trixie). Toy Story - Debian Wiki . **Architectures de Trixie (Debian 13)** : La version Trixie de Debian supporte: *amd64*, *aarch64*, *armhf*, *powerpc*, *riscv64* et *system z*. Debian Trixie Release Information . **Premier nom de code** : `Buzz` (Debian 1.1), annoncé le 17 juin 1996. Debian Releases . **Dernier nom de code annoncé** : La version 14 est `Forky` et la version 15 sera `Duke`. Annonce de Forky - Debian Mailing List

1.4. Installation préconfigurée (Question 5)

Attention
La modification de l'installation automatisée se fait exclusivement dans le fichier `preseed-fr.cfg`.

1.4.1. Ajustement de la pré-configuration

Question	Réponse
Différence `pkgsel/include` et `tasksel`	`d-i pkgsel/include` permet de spécifier une liste de **paquets individuels** précis à installer (ex. : `git`, `curl`). `tasksel tasksel/first` installe des **tâches entières** qui regroupent des centaines de paquets (ex. : `mate-desktop` ou `web-server`) [19].
Sécurité du preseed	Oui, le fichier preseed contient les mots de passe de `root` et de `user` en clair. En production, il faut protéger ce fichier en le stockant sur un serveur web interne sécurisé (HTTPS avec authentification), en restreignant drastiquement les droits de lecture du fichier physique ou en écrivant le hash uniquement [20].

1.4.2. Notre configuration automatisée

Pour répondre aux consignes de l'installation automatisée, nous avons édité le fichier `preseed-fr.cfg`. Voici les modifications clés apportées :

Installation de MATE :

```
d-i tasksel/first multiselect standard, mate-desktop
```

Ajout des paquets supplémentaires :

```
d-i pkgsel/include string sudo git sqlite3 curl bash-completion fastfetch
```

Ajout de l'utilisateur user dans root : Nous avons modifié le paramètre par défaut des groupe du

nouvel utilisateur pour y rajouter `sudo` :

```
-d-i passwd/user-default-groups string audio cdrom video
+d-i passwd/user-default-groups string audio cdrom video sudo
```

2. Semaine 8 : Comparatif Markdown vs AsciiDoc

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

2.1. Étude comparative des formats à balisage léger

Dans le cadre de notre projet, nous avons comparé deux langages de balisage léger très populaires : le Markdown et l'AsciiDoc. Voici un grand tableau comparatif détaillant leurs concepts, origines et syntaxes.

Sujet	Markdown (.md)	Asciidoctor (.adoc)
Description	Langage de balisage léger très populaire, conçu pour être facile à lire et à écrire. Très orienté pour le web et les plateformes collaboratives (ex: GitHub).	Langage de balisage léger plus riche et standardisé, conçu initialement pour la rédaction d'articles, de livres techniques et de documentations complexes.
Lien vers la documentation officielle	Site officiel Markdown	Documentation Asciidoctor
Origine (Date et Créateurs)	Créé en **2004** par John Gruber (avec l'aide d'Aaron Swartz).	Créé en **2002** par Stuart Rackham.

Comment convertir (sans extension) vers HTML et PDF	Utilisation classique avec la commande **Pandoc** :
	`pandoc fichier.md -o fichier.html`
	`pandoc fichier.md -o fichier.pdf`
		Utilisation avec la suite **Asciidoctor** :
		`asciidoctor fichier.adoc`
		`asciidoctor-pdf fichier.adoc`

Écrire en gras	Pour écrire en gras, il faut utiliser deux astérisques de chaque côté du mot : **mot**	
Écrire en italique	Pour écrire en italique, il faut utiliser un tiret bas : _mot_	
Écrire en gras ET italique	Pour combiner les deux, on cumule les symboles : **_mot_**	
Texte souligné	Le Markdown natif ne possède pas de syntaxe pour souligner. Il faut utiliser les balises HTML : <u> mot </u>	AsciiDoc gère le soulignement grâce aux rôles (les espaces autour sont importants) : [.underline]# mot #

<table>
<tr><th>Sujet</th><th>Markdown (.md)</th><th>Asciidoctor (.adoc)</th></tr>
<tr><td>Texte surligné</td><td>Cela dépend des extensions (parfois <code>== mot ==</code>), ou via la balise HTML : <code><mark>mot</mark></code></td><td>AsciiDoc utilise simplement le symbole dièse de chaque côté (les espaces autour sont importants) :
mot #</td></tr>
<tr><td>Texte barré</td><td>On utilise très souvent deux tildes :
~~ ~~mot~~ ~~</td><td>AsciiDoc utilise les rôles :
[.line-through]# ~~mot~~ #</td></tr>
<tr><td>Faire des listes à puces (avec sous-liste)</td><td>On utilise des astérisques, avec une indentation (espaces) pour le sous-niveau :
<pre>* Élément principal 1
* Élément principal 2
  * Sous-élément</pre></td><td>On utilise des astérisques, et on en ajoute un supplémentaire par niveau de profondeur :
<pre>* Élément principal 1
* Élément principal 2
** Sous-élément</pre></td></tr>
<tr><td>Faire des listes numérotées</td><td>On utilise des nombres suivis d’un point :
<pre>1. Première étape
2. Deuxième étape</pre></td><td>Asciidoc gère la numérotation automatiquement avec des simples points :
<pre>. Première étape
. Deuxième étape</pre></td></tr>
<tr><td>Créer un tableau simple</td><td>On le dessine avec des barres verticales (<code>|</code>) et des tirets pour séparer l’en-tête : <pre>| Colonne 1	Colonne 2

Valeur 1	Valeur 2 |</pre></td><td>On déclare le nombre de colonnes puis on liste les cellules les unes après les autres :
	<pre>[cols="1,1", options="header"]
	!===
	! Colonne 1 ! Colonne 2
	! Valeur 1 ! Valeur 2
	!===</pre>
	Table 1. Résultat :
	<table><tr><th>Colonne 1</th><th>Colonne 2</th></tr><tr><td>Valeur 1</td><td>Valeur 2</td></tr></table></td></tr>

</table>

<table>
<tr><th>Sujet</th><th>Markdown (.md)</th><th>Asciidoctor (.adoc)</th></tr>
<tr><td>Différents types de tableaux (tailles et fusions)</td><td>Très limité. Le Markdown classique ne gère ni les tailles de colonnes précises, ni la fusion de cellules (il faut coder en HTML).</td><td>Très puissant.
On ajuste la taille des colonnes avec <code>cols</code>. On fusionne des colonnes avec <code>2+</code> :

<pre>[cols="1,3"]
!===
2+! Titre fusionné sur 2 colonnes
! Case 1 petite ! Case 2 très large
!===</pre>
Table 2. Résultat :
<table><tr><td colspan="2">Titre fusionné sur 2 colonnes</td></tr><tr><td>Case 1 petite</td><td>Case 2 très large</td></tr></table></td></tr>
<tr><td>Insérer un lien hypertexte</td><td>Le texte est entre crochets, l’URL entre parenthèses :
[Debian](https://debian.org)</td><td>L’URL est au début, le texte est entre crochets à la fin :
https://debian.org[Debian]</td></tr>
<tr><td>Insérer une image</td><td>Un point d’exclamation, le texte alternatif, puis le lien :
![titreAlternatif](image/imageMarkdown.png)</td><td>La macro image, suivie des attributs (taille, alignement) :
<code>image::image/imageAdoc.png[titreAlternatif, width=50%]</code></td></tr>
<tr><td>Blocs d’informations (Admonitions : Notes, Avertissements…)</td><td>Le Markdown natif ne le gère pas. (Il faut utiliser des extensions spécifiques de Pandoc).</td><td>Intégré nativement avec des icônes.

<pre>[NOTE]
====
Ceci est une note.
(Les autres types sont TIP, WARNING, CAUTION, IMPORTANT)
====</pre>
Résultat :
Ceci est une note. (Les autres types sont TIP, WARNING, CAUTION, IMPORTANT)</td></tr>
</table>

<table>
<tr><th>Sujet</th><th>Markdown (.md)</th><th>Asciidoctor (.adoc)</th></tr>
<tr><td>Blocs de code avec coloration syntaxique</td><td>Trois accents graves suivis du langage :


```bash
echo "Bonjour"
```

</td><td>Directive source suivie du langage, puis délimiteurs <code>----</code> (on utilise <code>...</code>. pour imbriquer l'exemple sans casser le tableau) :


```
[source,bash]
....
echo "Bonjour"
....
```

</td></tr>
<tr><td>Citations (Quotes)</td><td colspan="2">On utilise le chevron vers la droite <code>></code> en début de ligne :

```
> Ceci est une citation de texte.
```

AsciiDoc offre aussi des blocs avancés pour les citations :

```
[quote, Auteur, Source]
____
Ceci est une citation avancée en AsciiDoc.
____
```

Résultat :

> Ceci est une citation avancée en AsciiDoc.
>
> — Auteur, Source</td></tr>
<tr><td>Notes de bas de page</td><td>On doit créer une ancre et son contenu :


```
Ceci est mon texte[^1].

[^1]: Et voici la note de bas de page.
```

</td><td>Tout se fait sur la même ligne avec la balise footnote :


```
Ceci est mon texte footnote:[Et voici la note directement intégrée ici].
```

</td></tr>
</table>

3. Semaine 10 : Étude des applications clientes (Git)

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

3.1. Configuration globale de Git

La commande `git config --global` [21] permet de définir des variables globales pour l'ordinateur, qui seront utilisées lors de vos commits (comme votre nom ou votre e-mail).
Elles ne sont évidemment à faire qu'une fois par machine, et pas à chaque projet. Si l'email est différent que celui du compte, les commits ne seront pas associés au profil.

3.2. Concepts de base (Question 1)

Question	Réponse justifiée
Différence entre Git et les logiciels comme GitHub/GitLab/Forgejo ?	Git est l'outil de contrôle de version local en ligne de commande, alors que GitHub/GitLab/Forgejo sont des forges logicielles collaboratives (des plateformes web) permettant d'héberger et de gérer les dépôts Git en ligne [22].
Qu'est-ce qu'un dépôt Git ? Où sont stockées les données d'un dépôt local ?	Un dépôt Git, souvent appelé « repo », est un emplacement de stockage pour les fichiers d'un projet et l'ensemble de l'historique de leurs modifications. Ces données sont stockées de façon masquée dans le dossier local `.git` [23].
Différence entre un commit, une branche et un tag ?	Un **commit** est un instantané contenant les différences de chaque fichier par rapport au commit précédent. Une **branche** est une ligne de développement indépendante (une suite de commits) [24]. Un **tag** est une étiquette figée collée sur un commit précis (souvent utilisée pour marquer les versions logicielles).
Qu'est-ce qu'un dépôt distant (remote) ? Comment lister les remotes configurés ?	Un dépôt distant est un dépôt accessible par le réseau (via Internet ou un réseau local d'entreprise). La commande `git remote -v` permet de lister ceux qui sont configurés sur le projet [25].
Différence entre git pull et git fetch ?	`git fetch` : récupère les données du dépôt distant vers le dépôt local (sans modifier vos fichiers de travail en cours). `git pull` : effectue un **fetch** puis fusionne directement ces nouvelles données avec la branche locale actuelle [26].

3.3. Protocoles réseau pour Git (Question 2)

Question	Réponse justifiée
Quels sont les protocoles supportés par Git pour accéder à un dépôt ?	• Protocole Local (accès au système de fichiers sans port) • Protocole Git • Protocole HTTP/HTTPS (mode intelligent et idiot) [29] • Protocole SSH [30]
Sur quels ports réseau fonctionnent ces protocoles ?	Le protocole Git fonctionne sur le port `9418`, HTTP sur le port `80`, HTTPS sur `443` et SSH sur le port `22` [27].
Comment configurer l'authentification SSH pour Git ?	Il faut générer une paire de clés SSH sur l'ordinateur avec la commande `ssh-keygen -t ed25519 -C "exemple@univ-lille.fr"`, puis copier le contenu de la clé publique générée pour l'ajouter dans les paramètres de sécurité du compte sur la forge logicielle [28].

3.4. Manipulation pratique de Git (Question 3)

Question	Réponse justifiée / Commandes
Créez un nouveau dépôt Git local. Quelle commande utilisez-vous ? Que se passe-t-il ?	La commande `git init` crée un sous-dossier `.git` qui initialise la structure d'un dépôt vide dans le répertoire actuel [31].

Question	Réponse justifiée / Commandes

Ajoutez plusieurs fichiers, faites au moins 3 commits. Utilisez git log pour visualiser l'historique.	
	`touch fichier1 fichier2 fichier3 fichier4`
	`git add fichier1`
	`git commit -m "ajout du premier fichier"`
	`git add fichier2`
	`git commit -m "ajout du second fichier"`
	`git add fichier3 fichier4`
	`git commit -m "ajout du fichier 3 et 4"`
	`git log`
	
	`user@S203:~/exemple$ git log`
	`commit be99f4166cf068d32233745e234c032dd82cc12b (HEAD -> master)`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:37 2026 +0100`
	
	`ajout du fichier 3 et 4`
	
	`commit 1cb9411aefaa8fd404589caa304307193154490e`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:53:10 2026 +0100`
	
	`ajout du second fichier`
	
	`commit ad4e99ad7d5e6aa8dc48a2c5df1662eeec81be1b`
	`Author: Kylian Plantard <kylian.plantard.etu@univ-lille.fr>`
	`Date: Fri Mar 6 13:52:50 2026 +0100`
	
	`ajout du premier fichier`
	
	Le résultat est une liste des commits du plus récent au plus ancien avec leurs identifiants (be99f4166...), l'auteur, la date et le message de commit [32].

Créez une nouvelle branche, faites des modifications, puis fusionnez (merge) cette branche avec la principale. Expliquez.	`user@S203:~/exemple$ git checkout -b exemple`
	`Basculement sur la nouvelle branche 'exemple'`
	`user@S203:~/exemple$ echo "dans une branche" > texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit -m "ajout du texte"`
	`[exemple 1ac7443] ajout du texte`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	`user@S203:~/exemple$ git checkout master`
	`Basculement sur la branche 'master'`
	`user@S203:~/exemple$ git merge exemple`
	`Mise à jour be99f41..1ac7443`
	`Fast-forward`
	`texte | 1 +`
	`1 file changed, 1 insertion(+)`
	`create mode 100644 texte`
	
	Une branche est une chaine de commits parallèle aux autres branches (ici `master` et `exemple`).
	La fusion (merge) permet de combiner les changements d'une branche dans une autre en rajoutant les commits de la branche source vers la branche cible [33].

Question	Réponse justifiée / Commandes

Qu'est-ce qu'un conflit de fusion ? Provoquez-en un et montrez comment le résoudre.	`user@S203:~/exemple$ git merge conflit`
	`Fusion automatique de texte`
	`CONFLIT (contenu) : Conflit de fusion dans texte`
	`La fusion automatique a échoué ; réglez les conflits et validez le résultat.`
	`user@S203:~/exemple$ nano texte`
	`user@S203:~/exemple$ git add texte`
	`user@S203:~/exemple$ git commit`
	`[master 8591824] Merge branch 'conflit'`
	
	Un conflit survient lorsque deux branches modifient le même fichier de façon différente.
	Lors de la fusion, Git met le processus en pause et demande une résolution manuelle (choisir le code à garder et supprimer les marqueurs de conflit) [34].
	Pour résoudre le conflit, on modifie le fichier puis on le `git add` pour le marquer comme résolu, avant de faire un commit de fusion.
	
	`<<<<<<< HEAD`
	`conflit 1`
	`=======`
	`conflit 2`
	`>>>>>>> conflit`

Utilisez git diff pour comparer deux commits. Expliquez la sortie.	`diff --git a/texte b/texte`
	`index 9d600a1..7255def 100644`
	`--- a/texte`
	`+++ b/texte`
	`@@ -1 +1 @@`
	`-dans une branche`
	`+conflit 1`
	
	Les premières lignes indiquent les fichiers comparés (`a/texte` et `b/texte`) et les méta-données de la comparaison. (`index 9d600a1..7255def`)
	Les lignes précédées d'un `-` en rouge sont des suppressions et celles avec un `+` en vert sont des ajouts.
	Les autres lignes affichées servent de contexte autour du code modifié.

Qu'est-ce que le fichier .gitignore ? Créez-en un pour ignorer les fichiers Java (.class, .jar).	C'est un fichier texte permettant de lister les motifs des fichiers/dossiers qui ne doivent pas être suivis et versionnés par Git [35].
	
	`*.class`
	`*.jar`

3.5. Travail collaboratif (Question 4)

Question	Réponse justifiée / Commandes
Création d'un dépôt privé partagé sur GitLab et clonage par chaque membre.	il faut faire `git clone git@gitlab-ssh.univ-lille.fr:notthedreamteam/exemple-sae203.git` [36].

Chaque membre crée une branche, ajoute un fichier, et pousse (push) sa branche.	Voici la suite de commandes utilisées pour la branche de chaque membre [40] :
	`git switch --create emilien`
	`echo "encore un fichier" > encore_un_fichier`
	`git add encore_un_fichier`
	`git commit -m "ajoute encore_un_fichier"`
	`git push --set-upstream origin emilien`

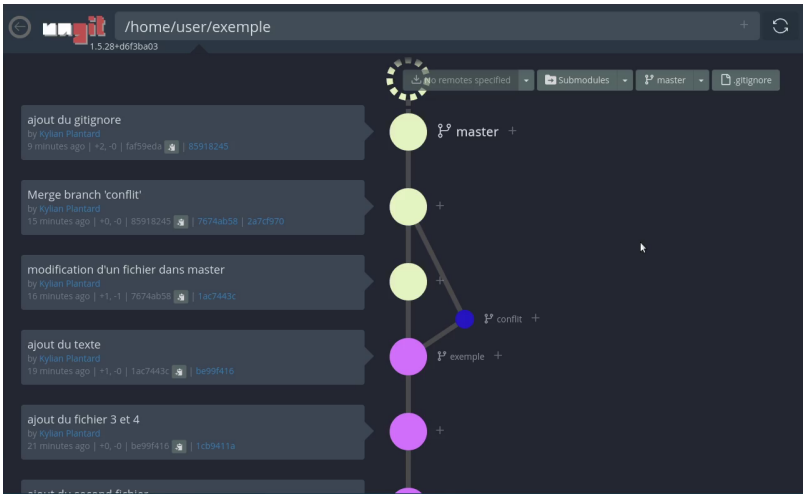
Fusion de toutes les branches dans la principale. Documentez les éventuels conflits.	On a fait des merge requests dans gitlab pour fusionner les branches [37]. Nous avons pas eu de conflits car nous avons pas modifié les meme fichiers.
Qu'est-ce qu'une pull request (ou merge request) ? À quoi sert-elle ?	Une pull/merge request sert a demander la fusion d'une branche source vers une branche destination tout en permettant aux autres membres de vérifier le code avant validation [38].
Testez la commande git blame. À quoi sert-elle ?	La commande `git blame <nom_du_fichier>` permet d'afficher le fichier ligne par ligne en indiquant pour chaque ligne l'auteur, la date et le hash du commit qui l'a modifiée pour la dernière fois. C'est très utile pour retrouver l'origine d'un bout de code ou d'un bug [39].

3.6. Les interfaces graphiques pour git (Question 5)

- **Qu'est-ce que le logiciel gitk ? Comment se lance-t-il ?**
  C'est un visualiseur graphique historique pour les dépôts Git. Il permet d'explorer l'historique des commits, l'arborescence des branches, et de lire les diffs plus confortablement. Il se lance simplement en tapant la commande `gitk` dans le terminal depuis un dépôt Git [41].
- **Qu'est-ce que le logiciel git-gui ? Comment se lance-t-il ?**
  C'est une interface graphique orientée vers la création de commits. Contrairement à gitk, il permet de sélectionner (stage) des fichiers, d'écrire le message de commit et de valider. Il se lance en tapant `git gui` dans le terminal [42].

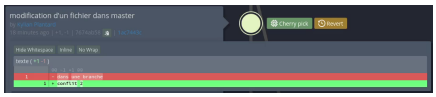
3.7. Installons autre chose et comparons (Questions 6 & 7)

Choix de l'interface graphique (Question 6)



- **Pourquoi avez-vous choisi ce logiciel ?**
  Nous avons choisi **Ungit** pour son approche extrêmement visuelle. Il représente l'historique de Git sous forme de nœuds interactifs et d'animations très intuitives, ce qui rend la compréhension des branches beaucoup plus claire que les vieux clients natifs [43].
- **Comment l'avez-vous installé ?**
  Ungit est un paquet Node.js. Après avoir installé Node.js et `npm` sur le système, nous l'avons installé via la commande globale : `sudo npm install -g ungit`. Ensuite, il se lance en tapant `ungit` et s'ouvre directement sous forme d'onglet dans le navigateur web.
- **Comparaison avec les outils inclus dans Git et la ligne de commande pure :**
  Par rapport à `gitk` ou `git-gui`, l'interface d'Ungit est bien plus moderne et unifie tout (historique, commit, push, merge) au même endroit de manière fluide. Par rapport à la ligne de commande pure, Ungit aide à mieux visualiser la topologie du projet, mais la ligne de commande reste plus rapide pour les opérations basiques et indispensable pour des commandes Git complexes.

Analyse comparative approfondie (Question 7)

Opération	En ligne de commande	Avec l'outil graphique choisi
Visualiser l'historique des commits	`git log` ou `git log --oneline --graph`	Il suffit d'ouvrir la page web et de scroller pour voir l'arbre interactif des nœuds colorés.
Créer une nouvelle branche	`git branch <nom>` ou `git switch -c <nom>`	Cliquer sur le nœud du commit actuel, taper le nom de la branche dans le champ de texte et cliquer sur le bouton "+ Branch".
Comparer deux versions d'un fichier (diff)	`git diff` ou `git diff <commit1> <commit2>`	

Opération	En ligne de commande	Avec l'outil graphique choisi
Résoudre un conflit de fusion	Ouvrir le fichier dans un éditeur (nano/vim), effacer les balises (<<<<<<<, =======), garder le bon code, puis faire `git add` et `git commit`.	Ungit signale le conflit en rouge ("Resolve conflicts"). On peut ouvrir un outil de diff visuel ou marquer le fichier comme résolu directement depuis l'interface après modification.

- **Quelles opérations sont plus rapides en ligne de commande ? Lesquelles sont plus claires avec l'interface graphique ?**
  Les opérations courantes, basiques et répétitives (comme le `git add`, la création de branche, un `git commit` simple ou un `git push`) sont beaucoup plus rapides à taper en ligne de commande. En revanche, comprendre l'arborescence des branches, visualiser un historique complexe et repérer les modifications dans plusieurs fichiers en même temps est bien plus clair avec l'interface graphique.
- **Dans un contexte professionnel, quel outil privilégieriez-vous et pourquoi ?**
  Dans un contexte professionnel, l'idéal est de combiner les deux. La ligne de commande est privilégiée pour la rapidité des actions quotidiennes et pour l'automatisation de scripts. L'interface graphique (Ungit, GitKraken, ou un outil intégré à l'IDE comme VS Code) est en revanche indispensable pour faire des revues de code, comprendre l'historique d'un projet avant une fusion (merge), et résoudre des conflits complexes avec moins de risque d'erreur.

4. Semaine 11 : Installation de Forgejo

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

4.1. Question 1 : À propos de Forgejo

Question : Qu'est-ce que Forgejo ? Qu'est-ce qu'un fork ? Retracez l'histoire de Forgejo. Qu'est-ce qui différencie sa gouvernance de celle de Gitea ?

Réponse et Explication :

- **Forgejo** : C'est une forge logicielle libre et auto-hébergée, permettant de gérer des dépôts Git, le suivi des bugs (issues), et l'intégration continue [44].
- **Fork** : En développement logiciel, un "fork" (ou embranchement) consiste à copier le code source d'un logiciel existant pour créer un nouveau projet indépendant, souvent suite à un désaccord technique ou éthique avec l'équipe d'origine [45].
- **Histoire** : L'histoire commence avec **Gogs**, une forge très légère écrite en Go. Gogs évoluant trop lentement au goût de certains contributeurs, ils ont forké le projet pour créer **Gitea**. Plus récemment, l'entreprise derrière Gitea a pris un virage commercial (création d'une entreprise à but lucratif, Gitea Ltd). En réaction, la communauté (notamment via l'association Codeberg) a forké Gitea pour créer **Forgejo** (qui signifie "forge" en Espéranto). Gogs et Gitea existent toujours.
- **Gouvernance** : La grande différence réside dans le fait que Gitea est désormais contrôlé par une entreprise commerciale, tandis que Forgejo garantit une gouvernance 100% communautaire et à but non lucratif, empêchant toute privatisation du code.

Commandes utiles (pour l'installation évoquée dans le cours) :

```
# Téléchargement du binaire officiel
wget -O forgejo
https://codeberg.org/forgejo/forgejo/releases/download/v1.21.0/forgejo-1.21.0-linux-
amd64
chmod +x forgejo
```

4.2. Question 2 : Architecture et protocoles réseau

Question : Par défaut, Forgejo écoute sur le port 3000. Pourquoi pas le port 80 (port standard HTTP) ?

Réponse et Explication :

Sous Linux, les ports inférieurs à 1024 (appelés ports privilégiés) nécessitent les droits d'administrateur (`root`) pour pouvoir être ouverts et écoutés [46]. Faire tourner une application web complexe comme Forgejo avec les droits `root` est une faille de sécurité majeure (si l'application est compromise, tout le serveur l'est). Forgejo tourne donc sous un utilisateur standard (souvent nommé `git` ou `forgejo`) et utilise un port non privilégié comme le 3000.

En production, on place généralement un "Reverse Proxy" (comme Nginx ou Apache) qui, lui, a les droits d'écouter le port 80 ou 443 et qui redirige le trafic de manière sécurisée vers le port 3000 local.

4.3. Question 3 : Base de données

Question : Pourquoi SQLite plutôt que PostgreSQL ou MySQL/MariaDB ? Où se trouve le fichier de base de données SQLite ? Qui en est le propriétaire ?

Réponse et Explication :

- **Choix de SQLite** : SQLite est une base de données orientée fichier, ultra-légère et ne nécessitant aucune configuration de serveur (pas de service supplémentaire à faire tourner ou à sécuriser). C'est parfait pour une SAÉ, un petit groupe ou un usage sur une machine virtuelle ayant peu de RAM (comme nos 2048 Mo) [47].
- **Emplacement et Propriétaire** : En général, le fichier se trouve dans le répertoire de travail de l'application, souvent dans `/var/lib/forgejo/data/forgejo.db`. Le propriétaire de ce fichier **doit** être l'utilisateur système qui exécute le service Forgejo (ex: `git` ou `forgejo`), afin qu'il puisse y lire et y écrire les données.

Commandes utiles :

```
# Vérifier l'emplacement et le propriétaire du fichier SQLite
ls -l /var/lib/forgejo/data/forgejo.db
```

4.4. Question 4 : Gestion du service systemd

Question : Qu'est-ce qu'un service systemd ? Expliquez le fichier de service. Différence entre start, enable et restart ? Comment visualiser les logs ?

Réponse et Explication :

- **Systemd** : C'est le gestionnaire de système et de services de Linux. Il permet de démarrer, arrêter et surveiller les processus fonctionnant en arrière-plan (les démons), et de s'assurer qu'ils redémarrent en cas de crash [48].
- **Le fichier de service (`/etc/systemd/system/forgejo.service`)** :
  - `[Unit]` : Contient les métadonnées et les dépendances (ex: "ne démarrer ce service qu'après le démarrage du réseau").
  - `[Service]` : Définit comment exécuter l'application (l'utilisateur cible `User=git`, le chemin vers le binaire `ExecStart=…`, le répertoire de travail, etc.).
  - `[Install]` : Définit à quel moment de la séquence de démarrage le service doit être lancé (généralement `multi-user.target`).
- **Commandes systemctl** :
  - `start` : Lance le service immédiatement.

- `enable` : Configure le service pour qu'il se lance automatiquement au démarrage de la machine.
- `restart` : Arrête puis redémarre le service (utile après un changement de configuration).

Commandes utiles :

```
# Gérer le service
sudo systemctl enable forgejo
sudo systemctl start forgejo
sudo systemctl restart forgejo

# Visualiser les logs en direct
sudo journalctl -u forgejo -f
```

4.5. Question 5 : Sécurité

Question : Examinez les permissions de `/etc/forgejo` et `app.ini`. Pourquoi sont-elles importantes ? Le serveur SSH écoute sur quel port ? Mesures pour un accès Internet ?

Réponse et Explication :

- **Permissions (`app.ini`)** : Le fichier `app.ini` contient des mots de passe en clair (clés secrètes, mail, configuration interne). Ses permissions doivent être très restrictives (généralement `0640` ou `0600`) pour que seul l'utilisateur `forgejo` (et `root`) puisse le lire [49].
- **Serveur SSH** : Le port 22 standard est déjà utilisé par le serveur SSH de la machine hôte Debian. Forgejo déploie donc son propre serveur SSH interne qui écoute sur un port alternatif (souvent le port `2222`).
- **Sécurité sur Internet** : Si le serveur était exposé, il faudrait impérativement :
  1. Configurer le HTTPS (certificats SSL/TLS via Let's Encrypt).
  2. Activer un pare-feu (`ufw`) pour ne laisser passer que les ports nécessaires (80, 443, 22, 2222).
  3. Installer `Fail2ban` pour bloquer les tentatives d'attaques par force brute.

Commandes utiles :

```
# Restreindre les permissions
sudo chmod 750 /etc/forgejo
sudo chmod 640 /etc/forgejo/app.ini
sudo chown -R forgejo:forgejo /etc/forgejo
```

4.6. Question 6 : Tests d'utilisation approfondis

Question : Projet privé vs public ? Ajouter des équipiers, lier une issue à un commit, tester clone/push en SSH.

Réponse et Explication :

- **Privé vs Public** : Un projet public est lisible (et clonable) par n'importe quel visiteur, même non authentifié. Un projet privé est invisible et inaccessible à moins d'être explicitement invité par le propriétaire.
- **Issues et Commits** : Forgejo permet de fermer automatiquement un ticket (issue) lors d'un push en utilisant des mots-clés spécifiques dans le message du commit (ex: `Fixes #1`).
- **Clé SSH** : Renseigner sa clé publique dans les paramètres Forgejo permet d'authentifier sa machine de manière sécurisée et d'éviter de taper son mot de passe à chaque `git push` [50].

Commandes utiles :

```
# 1. Créer sa clé SSH (si non existante)
ssh-keygen -t ed25519 -C "mon.email@exemple.com"

# 2. Cloner en utilisant le port SSH personnalisé (ex: 2222)
git clone ssh://git@localhost:2222/utilisateur/projet.git

# 3. Lier un commit à l'issue numéro 3
git commit -m "Correction du bug d'affichage. Fixes #3"
git push
```

4.7. Question 7 : Intégration et livraison continues (Pour aller plus loin)

Question : Qu'est-ce que CI/CD ? Workflow simple ? Avantages en équipe ?

Réponse et Explication :

- **CI (Continuous Integration)** : Automatisation des tests et de la vérification du code à chaque fois qu'un développeur pousse (push) des modifications. Cela garantit que le nouveau code ne "casse" pas l'application [51].
- **CD (Continuous Delivery/Deployment)** : Déploiement automatisé de l'application (ex: création du rapport PDF Asciidoc ou publication du site web) une fois les tests de la CI validés.
- **Avantages** : Réduction massive des bugs, retours immédiats pour les développeurs, standardisation de la compilation (plus de "ça marche sur ma machine mais pas sur le serveur"), et gain de temps sur les tâches répétitives.

Exemple d'un fichier Workflow Forgejo Actions (`.forgejo/workflows/build.yml`) :

```yaml
name: Compilation AsciiDoc
on: [push]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
```

```
      - name: Récupération du code
        uses: actions/checkout@v3
      - name: Installation Asciidoctor
        run: sudo apt-get install -y asciidoctor
      - name: Compilation HTML
        run: asciidoctor rapport.adoc
```


[1] Page de manuel usermod

[2] Page de manuel mount

[3] Wikipedia Processeur 64 bits

[4] Manuel VirtualBox

[5] Documentation Debian - Configuration matérielle requise

[6] Documentation VirtualBox - Mode NAT

[7] Page de manuel IP

[8] FAQ officielle des images CD Debian

[9] Wiki Debian - Environnements de bureau

[10] Wikipédia - Serveur mandataire (Proxy)

[11] Wiki Debian - Serveur Web

[12] Documentation VirtualBox - Redirection de ports NAT

[13] Page de manuel groups

[14] Page de manuel id

[15] Page de manuel su

[16] Page de manuel sudo

[17] Page de manuel uname

[18] Wiki Debian - Additions Invité VirtualBox

[19] Wiki Debian - Tasksel

[20] Manuel d'installation Debian - Sécurité de la pré-configuration

[21] Documentation git config

[22] Documentation officielle Git - Dépôts distants

[23] Documentation Git - Le répertoire Git

[24] Documentation Git - Les Branches

[25] Documentation Git - Commandes Remote

[26] Documentation Git - Fetch vs Pull

[27] Documentation Git - Les protocoles réseau

[28] Documentation Git - Génération clé SSH

[29] Le mode intelligent est un serveur http qui comprend l'authentification et les push, tandis que le mode idiot est juste un serveur http basique qui distribue le dossier `.git`.

[30] Livre Git - Les Protocoles

[31] Documentation Git - git init

[33] Documentation Git - git merge

[34] Documentation Git - Résolution de conflits

[35] Documentation Git - gitignore

[36] Documentation GitLab - Clonage d'un dépôt

[37] Documentation GitLab - Merge Requests

[38] Documentation GitLab - Merge Requests

[39] Documentation officielle Git - git blame

[40] Documentation Git - Les branches

[41] Documentation officielle Git - gitk

[42] Documentation officielle Git - git-gui

[43] Dépôt officiel GitHub de Ungit

[44] FAQ officielle de Forgejo
[45] Wikipédia - Fork (développement logiciel)
[46] W3C - Architecture des ports privilégiés sous Linux
[47] Documentation officielle SQLite
[48] Wiki Debian - systemd
[49] Wiki Debian - Permissions des fichiers
[50] Documentation Git - Génération des clés SSH
[51] GitLab - Guide sur la CI/CD